

CPSC471-Network Communications IEEE

Full Project Report

Client-to-Server File Transfer using Sockets

Authors: Bryan Tran | Peter Chau

Affiliation: California State University, Fullerton

Course: CPSC 471 Network Communications

Programming Language: Python

Date: December 5, 2025

Abstract

This project implements a simple Python-based client–server file transfer system using TCP sockets. The objective was to create a functional simulation of a basic FTP-like protocol capable of listing server files, uploading files to the server, downloading files from the server, and supporting multiple simultaneous clients. The server utilizes multithreading to maintain concurrent connections and has been tested both locally and remotely, including deployment on an AWS cloud instance. Python’s standard socket module served as the core technology for communication, providing the foundational API for TCP networking [1]. Testing confirmed stable multi-client operation, successful file transfers, and correct command processing. This project demonstrates how Python enables rapid development of networked applications while providing a foundational understanding relevant to real-world distributed systems.

Keywords

Python, Computer Networks, Socket Programming, Protocol Simulation, Client–Server Systems, File Transfer, Multithreading, Computer Networks

1. Introduction

1.1 Background

Client–server communication is a foundational concept in computer networks, enabling distributed systems to exchange data reliably. TCP sockets provide a persistent and reliable communication channel, and Python’s built-in socket library offers an accessible API for creating networked applications [1]. This project implements a simplified file transfer system using this library, which supports multiple clients simultaneously through multithreading. Clients can perform operations such as listing files, retrieving files, and uploading new files.

1.2 Objective

- Develop a port-based client–server communication system.
- Implement basic file transfer operations (ls, get, put, exit)
- Provide support for multiple clients via multithreading
- Host the server remotely on the AWS cloud

1.3 Scope

- TCP client-server communication
 - File upload (put), download (get), and listing (ls) functionality
 - Multi-client support using threading
-

2. Related Work

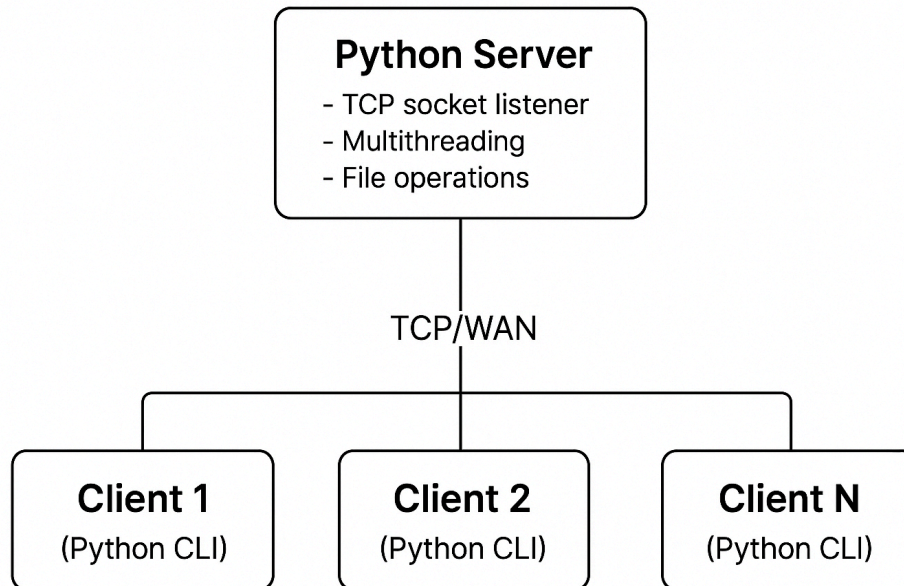
Existing FTP implementations, such as FileZilla or Python’s ftplib module, provide comprehensive features for file transfer. Although we cannot directly replicate it, our project simplifies these concepts for the purpose of demonstrating socket programming fundamentals. The Python socket library was used for TCP communication, and threading was used to manage multiple clients simultaneously.

3. System Design and Architecture

3.1 System Overview

The system uses a standard client–server architecture. The server listens on a TCP port and spawns a thread for each new client. Clients maintain an active session with the server and send commands, which are acknowledged by both the clients and the server.

3.2 Architecture Diagram



- Python Server will be hosted on an AWS Bucket. Once active, it will open a port and be listening for any inbound TCP requests
- Clients 1..2..N are independent and located on separate networks, connecting via the AWS Bucket NAT IP and port number for the server
- TCP connection travels through the internet to provide clients access to the server located on the cloud

3.3 Modules

- **Client Module:** Connects to the server, sends commands, and handles file reads/writes.
 - **Server Module:** Listens on TCP, accepts client connections, and assigns threads.
 - **Protocol Module:** simple command-response operations using plain-text commands.
-

4. Implementation

4.1 Technologies Used

- Python 3.x: Programming language
- Socket library: TCP client-server communication

- Threading library: Concurrent client handling
- OS library: File system operations
- Amazon Web Services: Hosts server on the cloud for remote testing

4.2 Key Features

- Establishes TCP connection between client and server
- Supports multi-client communication
- Implements ls, get, put, and exit commands
- Simple file storage and retrieval on both server and client

4.3 Code Overview

This section of client.py implements an FTP client that connects to a server at a user-specified IP address and port 5000. It continuously prompts the user for commands (ls, get <filename>, put <filename>) and sends them to the server. The client receives responses from the server and handles them accordingly: ls displays the list of files on the server, get downloads a specified file if available, and put uploads a file from the client to the server. Basic input validation and error handling ensure that commands are properly formatted and files exist before transfer.

```

1 import socket
2 import os
3
4 HOST = input("Enter server IP (or 'localhost'): ").strip()
5 PORT = 5000
6
7 client = socket.socket()
8 client.connect((HOST, PORT))
9 print(f"Connected to server at {HOST}:{PORT}")
10
11 while True:
12     command = input("ftp> ").strip()
13
14     if not command:
15         continue
16
17     client.send(command.encode())
18     parts = command.split()
19     cmd = parts[0]
20
21     if cmd == 'ls':
22         data = client.recv(4096).decode()
23         print("Server files:\n" + data)
24
25     elif cmd == 'get':
26         if len(parts) < 2:
27             print("Usage: get <filename>")
28             continue
29
30         filename = parts[1]
31
32         status = client.recv(1024).decode()
33
34         if status == 'OK':
35             data = client.recv(4096)
36             with open(filename, 'wb') as f:
37                 f.write(data)
38             print(f"Downloaded '{filename}' successfully.")
39         else:
40             print(status)
41
42     elif cmd == 'put':
43         if len(parts) < 2:
44             print("Usage: put <filename>")
45             continue
46
47         filename = parts[1]

```

This section of `server.py` defines a multi-client FTP server that listens on all network interfaces (0.0.0.0) at port 5000. The `handle_client` function manages each client connection in a separate thread, allowing multiple clients to interact with the server concurrently. It processes commands sent by clients—`ls` lists files in the server's directory, `get <filename>` sends a requested file to the client if it exists, and `put <filename>` receives a file from the client and saves it. Each command includes basic error handling, such as checking for missing filenames or non-existent files, and logs activity for each client connection.

```

import socket
import os
import threading

HOST = '0.0.0.0'
PORT = 5000

# Function to handle each client separately and to be called by threads
def handle_client(conn, addr):
    print(f"New connection from {addr}")

    while True:
        try:
            data = conn.recv(1024).decode()
            if not data:
                break

            print(f"Client command: {data}")
            parts = data.split()
            command = parts[0]

            if command == 'ls':
                files = os.listdir('.')
                file_list = '\n'.join(files)
                conn.send(file_list.encode())

            elif command == 'get':
                if len(parts) < 2:
                    conn.send(b'ERROR: No filename provided')
                    continue

                filename = parts[1]

                if os.path.exists(filename):
                    conn.send(b'OK')
                    with open(filename, 'rb') as f:
                        conn.sendall(f.read())
                    print(f"Sent file '{filename}' to {addr}")
                else:
                    conn.send(b'ERROR: File not found')

            elif command == 'put':
                if len(parts) < 2:
                    conn.send(b'ERROR: No filename provided')
                    continue

                filename = parts[1]

```

4.4 Algorithms

- TCP client-server communication
- Threaded client handling to support multiple simultaneous connections
- File reading/writing operations for get and put

5. Results and Discussion

5.1 Testing Scenarios

- LS Command: Lists files on server correctly
- GET Command: Downloads file to client's clientFiles/ directory
- PUT Command: Uploads file to server directory
- Multiple Clients: Successfully handled using threading

5.2 Performance Metrics

- Latency and throughput are acceptable for small text files (<1 MB)
- No packet loss observed on local network testing

5.3 Challenges and Resolutions

- Multiple client handling: Solved using threading.Thread
- File overwriting during put: Added clear directory structure (clientFiles/)

5.4 Discussion

- Strengths: Easy-to-understand protocol, works locally, supports multiple clients
 - Limitations: Not secure, handles only small files, no directory management
-

6. Conclusion and Future Work

6.1 Conclusion

- The project successfully demonstrates a simplified FTP client-server model using Python. It highlights fundamental networking concepts, socket programming, and multi-threading in a practical setting.

6.2 Future Work

- Add file encryption and authentication

- Support large files and binary transfers
- Implement additional FTP commands and directory navigation for scaling purposes

7. References

[1] Python Software Foundation, “*socket — Low-level networking interface*,” Python Documentation. Accessed: Dec. 4, 2025. [Online].

[2] GeeksforGeeks, “Socket Programming in Python,” GeeksforGeeks, Jun. 20, 2017. <https://www.geeksforgeeks.org/python/socket-programming-python/>

8. Appendices

server.py

```

1 import socket
2 import os
3
4 # Server setup
5 HOST = 'localhost'
6 PORT = 5000
7
8 server = socket.socket()
9 server.bind((HOST, PORT))
10 server.listen(1)
11 print(f"Server listening on {HOST}:{PORT}...")
12
13 conn, addr = server.accept()
14 print(f"Connected by {addr}")
15
16 while True:
17     # Receive command
18     data = conn.recv(1024).decode()
19     if not data:
20         break
21
22     print(f"Client command: {data}")
23     parts = data.split()
24     command = parts[0]
25
26     # ---- LS COMMAND ----
27     if command == 'ls':
28         files = os.listdir('.')
29         file_list = '\n'.join(files)
30         conn.send(file_list.encode())
31
32     # ---- GET COMMAND ----
33     elif command == 'get':
34         if len(parts) < 2:

```

```

root@vsnikku:/mnt/c/Users/crazy/OneDrive/Documents/GitHub/CFSC-471-Network-Programming# cd p1-group8
root@vsnikku:/mnt/c/Users/crazy/OneDrive/Documents/GitHub/CFSC-471-Network-Programming/p1-group8# ls
README.md client.py server.py test1.txt test2.txt
root@vsnikku:/mnt/c/Users/crazy/OneDrive/Documents/GitHub/CFSC-471-Network-Programming/p1-group8# python3 p1-group8
python3: can't open file '/mnt/c/Users/crazy/OneDrive/Documents/GitHub/CFSC-471-Network-Programming/p1-group8/p1-group8': [Errno 2] No such file or
directory
root@vsnikku:/mnt/c/Users/crazy/OneDrive/Documents/GitHub/CFSC-471-Network-Programming/p1-group8# ls
README.md client.py server.py test1.txt test2.txt
root@vsnikku:/mnt/c/Users/crazy/OneDrive/Documents/GitHub/CFSC-471-Network-Programming/p1-group8# python3 server.py
Server listening on localhost:5000...
Connected by ('127.0.0.1', 36888)
Client command: ls
Sent file 'test1.txt' to client.

```

client.py

```
1 import socket
2 import os
3
4 # Setup client
5 HOST = 'localhost'
6 PORT = 5000
7
8 client = socket.socket()
9 client.connect((HOST, PORT))
10 print(f"Connected to server at {HOST}:{PORT}")
11
12 while True:
13     command = input("ftp> ").strip()
14
15     if not command:
16         continue
17
18     client.send(command.encode())
19
20     parts = command.split()
21     cmd = parts[0]
22
23     # ---- LS COMMAND ----
24     if cmd == 'ls':
25         data = client.recv(4096).decode()
26         print("Server files:\n" + data)
27
28     # ---- GET COMMAND ----
29     elif cmd == 'get':
30         if len(parts) < 2:
31             print("Usage: get <filename>")
32             continue
33         filename = parts[1]
34         status = client.recv(1024).decode()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
root@kali:~/CPSC-471-Network-Programming/p1-group8# ls
client.py  README.md  server.py  test1.txt  test2.txt
root@kali:~/CPSC-471-Network-Programming/p1-group8# python3 client.py
Connected to server at localhost:5000
ftp> ls
Server files:
client.py
README.md
server.py
test1.txt
test2.txt
ftp> get test1.txt
Downloaded 'test1.txt' successfully.
ftp>
```

LS command:

```
Client command: ls
Server files:
test1.txt
test2.txt
ftp>
```

Client's visibility only within the server's database, displaying all existing files within the 'serverDatabase' directory

GET command:

```
Client command: get test1.txt
Sent file 'test1.txt' to client.
ftp> get test1.txt
Downloaded 'test1.txt' successfully to client's import folder
ftp>
```

Client is able to grab 'test1.txt' from the server's database and download it into the client's 'clientFiles' folder

PUT command:

```
CPSC-471-Netw...  Client command: put test2.txt
                  Received file 'test2.txt' from client and saved to server's databas
                  g
p1-group8         |
clientFiles       |
test1.txt         | U
test2.txt         | U
serverDataba...  |
test1.txt         | U
test2.txt         | U

ftp> put test2.txt
Uploaded 'test2.txt' successfully.
ftp>
```

Client is able to send its own file, 'test2.txt', across the socket connection, which is then verified by the server and then stored in the server's database

EXIT command:

```
CPSC-471-Netw...  Client command: exit
                  Server closed.
p1-group8         | PS C:\Users\pette\OneDrive\Documents\CSUF\Fall Semester 2025\CPSC 4
                  71 - Computer Communications\CPSC-471-Network-Programming\p1-group8
clientFiles       | >
test1.txt         | U
test2.txt         | U
serverDataba...  |
test1.txt         | U
test2.txt         | U

ftp> exit
Goodbye!
Connection closed.
PS C:\Users\pette\OneDrive\Documents\CSUF\Fall Semester 2025\CPSC 4
71 - Computer Communications\CPSC-471-Network-Programming\p1-group8
>
```

Once the client terminates the connection, the server subsequently sends a final 'Goodbye!' message and closes its connection alongside the client
server.py on AWS

The image shows a VS Code editor window with a dark theme. The top toolbar includes icons for Explorer, Search, Source Control, Run and Debug, and Extensions, along with a search bar containing 'CPSC-471-Network-Programming'. The Explorer sidebar on the left shows a file tree with 'test1.txt', 'test2.txt', 'client.py', and 'server.py'. The main editor area displays the content of 'client.py', which is a Python script for a client. The script imports 'socket' and 'os', prompts the user for a server IP (defaulting to 'localhost') and a port (5000), connects to the server, and enters a loop to handle commands like 'ls' and 'get'. The bottom panel contains a 'TERMINAL' tab showing a shell session. The session starts with a warning about an inaccessible identity file, followed by an attempt to connect to '18.217.173.237'. After a permission denied error, the user navigates to the 'p2-group8' directory and runs 'python3 server.py', which outputs 'Server listening on 0.0.0.0:5000'. The status bar at the bottom indicates the file is at line 65, column 1, with 4 spaces, using UTF-8 encoding and CRLF line endings, and is a Python file.

```
p2-group8 > client.py > ...
1 import socket
2 import os
3
4 HOST = input("Enter server IP (or 'localhost'): ").strip()
5 PORT = 5000
6
7 client = socket.socket()
8 client.connect((HOST, PORT))
9 print(f"Connected to server at {HOST}:{PORT}")
10
11 while True:
12     command = input("ftp> ").strip()
13     if not command:
14         continue
15
16     client.send(command.encode())
17     parts = command.split()
18     cmd = parts[0]
19
20     if cmd == 'ls':
21         data = client.recv(4096).decode()
22         print("Server files:\n" + data)
23
24     elif cmd == 'get':
25         if len(parts) < 2:
26             print("Usage: get <filename>")
```

user@18.217.173.237:~
Warning: Identity file /home/ec2-user/.ssh/p2-group8.pem not accessible: No such file or directory.
The authenticity of host '18.217.173.237 (18.217.173.237)' can't be established.
ED25519 key fingerprint is SHA256:exEznV1zf+6Ddt0zhz+L595zeSsa05HsJhY+XNLx87I.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.217.173.237' (ED25519) to the list of known hosts.
ec2-user@18.217.173.237: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
Connection closed
[ec2-user@ip-172-31-41-232 ~]\$ ls
p2-group8
[ec2-user@ip-172-31-41-232 ~]\$ python3 server.py
python3: can't open file '/home/ec2-user/server.py': [Errno 2] No such file or directory
[ec2-user@ip-172-31-41-232 ~]\$ ls
p2-group8
[ec2-user@ip-172-31-41-232 ~]\$ cd p2-group8
[ec2-user@ip-172-31-41-232 p2-group8]\$ ls
README.md client.py server.py test1.txt test2.txt
[ec2-user@ip-172-31-41-232 p2-group8]\$ python3 server.py
Server listening on 0.0.0.0:5000

client.py on AWS

The image shows a Visual Studio Code editor window with the title bar "CPSC-471-Network-Programming". The editor has several tabs open: "test1.txt p2-group8", "test2.txt p2-group8", "client.py p2-group8", "test1.txt Part 1", and "server.py". The "client.py" tab is active, displaying the following Python code:

```
1 import socket
2 import os
3
4 HOST = input("Enter server IP (or 'localhost'): ").strip()
5 PORT = 5000
6
7 client = socket.socket()
8 client.connect((HOST, PORT))
9 print(f"Connected to server at {HOST}:{PORT}")
10
11 while True:
12     command = input("ftp> ").strip()
13     if not command:
14         continue
15
16     client.send(command.encode())
17     parts = command.split()
18     cmd = parts[0]
19
20     if cmd == 'ls':
21         data = client.recv(4096).decode()
22         print("Server files:\n" + data)
23
24     elif cmd == 'get':
25         if len(parts) < 2:
26             print("Usage: get <filename>")
```

Below the editor is a terminal window with the following output:

```
Output will be saved to ./response.xml. If you want to run parallel
executions in the same directory, define RTFILEPREFIX in the environment.
Doing so may significantly increase files generated.

Requires: curl, xmlint, GNU core utilities.
Optional: xsltproc for output formatting, gpg for encrypted credential.
Optional: wget as alternate for curl when not available.
root@Kermikyu:/mnt/c/Users/crazy/OneDrive/Documents/GitHub/CPSC-471-Network
-Programming/p2-group8# ls
README.md client.py server.py test1.txt test2.txt
root@Kermikyu:/mnt/c/Users/crazy/OneDrive/Documents/GitHub/CPSC-471-Network
-Programming/p2-group8# python3 client.py
Enter server IP (or 'localhost'): 18.217.173.237
Connected to server at 18.217.173.237:5000
ftp> ls
Server files:
client.py
README.md
server.py
test1.txt
test2.txt
ftp> get test1.txt
Downloaded 'test1.txt' successfully.
ftp> put test2.txt
Uploaded 'test2.txt' successfully.
ftp>
```

The terminal window also shows a list of open ports on the right side, all labeled "wsl".

client.py interacting with server.py on AWS

The image shows a Visual Studio Code editor window with a search bar at the top containing "CPSC-471-Network-Programming". The editor has several tabs open: "test1.txt p2-group8", "test2.txt p2-group8", "client.py p2-group8" (which is the active tab), "test1.txt Part 1", and "server.py".

The "client.py" file contains the following Python code:

```
1 import socket
2 import os
3
4 HOST = input("Enter server IP (or 'localhost'): ").strip()
5 PORT = 5000
6
7 client = socket.socket()
8 client.connect((HOST, PORT))
9 print(f"Connected to server at {HOST}:{PORT}")
10
11 while True:
12     command = input("ftp> ").strip()
13     if not command:
14         continue
15
16     client.send(command.encode())
17     parts = command.split()
18     cmd = parts[0]
19
20     if cmd == 'ls':
21         data = client.recv(4096).decode()
22         print("Server files:\n" + data)
23
24     elif cmd == 'get':
25         if len(parts) < 2:
26             print("Usage: get <filename>")
```

The bottom panel of the editor shows the "TERMINAL" output. It displays the execution of the script on an EC2 instance:

```
ED25519 key fingerprint is SHA256:exEznV1zF+6Ddt0zhz+LS95ze5sa05HsJhX+XNLx8
71.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.217.173.237' (ED25519) to the list of known
hosts.
ec2-user@18.217.173.237: Permission denied (publickey,gssapi-keyex,gssapi-u
ith-mic).
Connection closed
[ec2-user@ip-172-31-41-232 ~]$ ls
p2-group8
[ec2-user@ip-172-31-41-232 ~]$ python3 server.py
python3: can't open file '/home/ec2-user/server.py': [Errno 2] No such file
or directory
[ec2-user@ip-172-31-41-232 ~]$ ls
p2-group8
[ec2-user@ip-172-31-41-232 ~]$ cd p2-group8
[ec2-user@ip-172-31-41-232 p2-group8]$ ls
README.md client.py server.py test1.txt test2.txt
[ec2-user@ip-172-31-41-232 p2-group8]$ python3 server.py
Server listening on 0.0.0.0:5000
ssh -i ~/.ssh/p2-group8.pem ec2-user@your-ip>
New connection from ('137.151.175.80', 7580)
Sent file 'test1.txt' to ('137.151.175.80', 7580)
Received file 'test2.txt' from ('137.151.175.80', 7580)
Connection closed: ('137.151.175.80', 7580)
```

The status bar at the bottom indicates the file is at "Ln 65, Col 1", uses "UTF-8" encoding, and is a "Python" file.

AWS activated

Cloud Computing Services - A

Instance details | EC2 | us-

Instance details | EC2 | us-east

us-east-2.console.aws.amazon.c...

For quick access, place your bookmarks here on the bookmarks bar. [Import bookmarks no...](#)

All Bookmarks

aws

United States (Ohio)

Account ID: 8342-2107-6082

btran299

EC2 > Instances > i-057aa85a03f2ebbe9

EC2

<

Dashboard

EC2 Global View

Events

▼ Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

▼ Images

AMIs

AMI Catalog

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

▼ Network & Security

Security Groups

Elastic IPs

Placement Groups

Instance summary for i-057aa85a03f2ebbe9 (p2-group8-server) [Info](#)

Connect

Instance state

Actions

Updated 22 minutes ago

Instance ID

i-057aa85a03f2ebbe9

Private IPv4 addresses

172.31.41.232

Instance state

Running

Hostname type

IP name: ip-172-31-41-232.us-east-2.compute.internal

Instance type

t3.micro

Auto-assigned IP address

18.217.173.237 [Public IP]

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendations.

[Learn more](#)

Public IPv4 address

18.217.173.237 | [open address](#)

IPv6 address

–

Public DNS

ec2-18-217-173-237.us-east-2.compute.amazonaws.com | [open address](#)

Private IP DNS name (IPv4 only)

ip-172-31-41-232.us-east-2.compute.internal

Answer private resource DNS name

IPv4 (A)

Elastic IP addresses

–

VPC ID

vpc-08b2bb8de1972cc80

IAM Role

–

CloudShell

Feedback

Console Mobile App

Privacy

Terms

Cookie preferences

© 2025, Amazon Web Services, Inc. or its affiliates.